

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

二分算法

Binary Search and Binary Search on Answer

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 30 日

目录

1. 二分查找基础
2. 二分查找变体
3. 浮点数二分
4. 二分答案
5. 总结



目录

1 二分查找基础

2 二分查找变体

3 浮点数二分

4 二分答案

5 总结

什么是二分查找

二分查找：在**有序**数组中，每次将搜索区间缩小一半，直到找到目标元素。

经典生活类比：猜数字

- 数字在 1 到 100 之间，每次猜测后告诉你“大了”还是“小了”。
- 每次都猜中间的数字（50、25、75 ...），最多 7 次就能猜中。
- 这就是二分：每次砍掉一半的错误选项。

前置条件

- 数据必须具有**单调性**（通常要求已排好序）。
- 操作必须能通过一次比较，排除当前一半的区间。

二分的核心：利用单调性，每次排除一半，把 $O(n)$ 变成 $O(\log n)$

例题：查找目标元素

题目：给定一个升序排列的数组 `a[1..n]` 和一个目标值 `k`。请找到数组中等于 `k` 的元素的下标。如果存在多个，返回任意一个即可；如果不存在，输出 -1。

示例 1

- 输入：n = 5, k = 4, 数组为 [1, 3, 4, 6, 7]
- 输出：3（4 在下标 3 的位置）

示例 2

- 输入：n = 4, k = 5, 数组为 [1, 3, 4, 6]
- 输出：-1（找不到 5）

查找目标元素：解法

思路

- 维护区间 `[l, r]`，初始 `l = 1, r = n`。
- 每次取 `mid = (l + r) / 2`。
- 如果 `a[mid] == k`，直接返回下标。
- 如果 `a[mid] > k`，目标在左边，更新 `r = mid - 1`。
- 如果 `a[mid] < k`，目标在右边，更新 `l = mid + 1`。

参考代码

查找目标元素：解法


```cpp
int l = 1, r = n, best = -1;
while (l <= r) {
    int mid = (l + r) >> 1;
    if (a[mid] == k) {
        best = mid;
        break;
    } else if (a[mid] > k) {
        r = mid - 1;
    } else {
        l = mid + 1;
    }
}
cout << best << endl;
```

目录

1 二分查找基础

2 二分查找变体

3 浮点数二分

4 二分答案

5 总结

例题：查找第一个大于等于 k 的元素

题目：给定一个升序排列的数组 `a[1..n]` 和一个目标值 `k`。请找到数组中**第一个大于等于** `k` 的元素的下标。

示例 1

- 输入：$n = 5$, $k = 4$, 数组为 $[1, 3, 5, 7, 9]$
- 输出：3（5 在下标 3 的位置，是第一个 ≥ 4 的数）

示例 2

- 输入：$n = 5$, $k = 8$, 数组为 $[1, 3, 5, 7, 9]$
- 输出：5（9 在下标 5 的位置，是第一个 ≥ 8 的数）

示例 3

- 输入：$n = 5$, $k = 10$, 数组为 $[1, 3, 5, 7, 9]$
- 输出：-1（没有大于等于 10 的数）

查找第一个大于等于 k 的元素：解法

思路：用 best 记录可能的答案

- 当 `a[mid] >= k` 时，说明 `mid` 可能是一个答案，将其存入 `best`。
- 但我们要找的是“第一个”，所以还要继续向左找，即 `r = mid - 1`。
- 当 `a[mid] < k` 时，说明中值太小，只可能向右找，即 `l = mid + 1`。

参考代码

```
int l = 1, r = n, best = -1;
while (l <= r) {
    int mid = (l + r) >> 1;
    if (a[mid] >= k) {
        best = mid;
        r = mid - 1;   // 继续向左收缩
    } else {
        l = mid + 1;
    }
}
```

查找第一个大于等于 k 的元素：解法


```
11 cout << best << endl;
```

注意：这个模板常被称为“左边界二分”，是 C++ STL 中 `lower_bound` 的底层实现。

目录

1 二分查找基础

2 二分查找变体

3 浮点数二分

4 二分答案

5 总结

浮点二分与整数二分的区别

整数二分：答案在离散的整数集合中，通过 `mid + 1` 或 `mid - 1` 避开死循环。

浮点二分：答案在连续的实数域中。

- 边界直接写成 `l = mid` 和 `r = mid`。
- 无法判断“相等”，所以用区间长度 `r - l` 小于某个极小值作为终止条件。
- 这个极小值称为精度，通常用全局常量 `eps` 表示，取 `1e-6` 或 `1e-7`。
- **核心前提**：目标函数或判定条件在实数区间内必须具有**单调性**。

两种常见的终止条件

- `while (r - l > eps)`：当区间足够小时停止。
- `for (int i = 0; i < 100; i++)`：固定循环 100 次，通常比 eps 更安全，不会因为精度问题死循环。

浮点二分靠精度控制，题目必定隐含连续区间的单调性

例题：求平方根

题目：给定一个非负实数 x，求它的算术平方根 $\sqrt{x}$，保留 6 位小数。**单调性提示**：由于函数 $y = x^2$ 在 $x \geq 0$ 时**严格单调递增**，可以用二分逼近目标值。

示例 1

- 输入：x = 2
- 输出：1.414214

示例 2

- 输入：x = 9
- 输出：3.000000

求平方根：解法

核心思路：在 $[0, x]$ 内二分查找一个数，使得它的平方无限逼近 x。

关键步骤

- 边界设为 `l = 0, r = x`。
- 循环条件 `while (r - l > eps)`。
- 中间值 `mid = (l + r) / 2`。
- 若 `mid * mid > x`，说明取大了，右边界左移 `r = mid`。
- 否则取小了，左边界右移 `l = mid`。

为什么正确：因为平方函数单调递增，若 $\text{mid}^2 > x$，真实的根必然在 mid 的左侧；反之在右侧。每次缩小区间，最终夹逼出真实结果。

不断缩小范围，直到区间长度小于精度

求平方根：代码


```cpp
const double eps = 1e-7;  // 全局定义精度常量 *

double mySqrt(double x) {
    double l = 0, r = x;
    while (r - l > eps) {
        double mid = (l + r) / 2;
        if (mid * mid > x) {
            r = mid;          // 取大了，缩小右边界
        } else {
            l = mid;          // 取小了，扩大左边界
        }
    }
    return l;
}
```

例题：解方程求根

题目：给定方程 $f(x) = x^3 - 5x^2 + 10x - 80 = 0$。已知函数 $f(x)$ 在区间 $[0, 100]$ 上严格单调递增，且存在唯一实数根。求这个根（保留 6 位小数）。

单调性分析

- 因为题目已经保证了单调递增，所以当 $f(\text{mid}) > 0$ 时，根一定在 mid 的左侧。
- 当 $f(\text{mid}) < 0$ 时，根一定在 mid 的右侧。

示例

- 输出：5.72（实际计算出的近似值）

解方程求根：解法

核心思路：利用函数在区间内的单调性，通过判断 $f(\text{mid})$ 的符号来收缩边界。

关键步骤

- 设定边界 `l = 0, r = 100`（前提是根在这个区间内）。
- 循环 `while (r - l > eps)`。
- `mid = (l + r) / 2`。
- 若 `f(mid) > 0`，说明根在左侧，`r = mid`。
- 若 `f(mid) < 0`，说明根在右侧，`l = mid`。

浮点数比较：有时也会加上 `fabs(f(mid)) < eps` 作为额外终止条件，但如果区间长度足够小，直接靠 `r - l > eps` 控制也是标准写法。

单调性是二分求根的灵魂

解方程求根：代码


```cpp
#include <cmath>

const double eps = 1e-7;  // 全局定义精度常量 *

double f(double x) {
    return x * x * x - 5 * x * x + 10 * x - 80;
}

double solve() {
    double l = 0, r = 100;
    while (r - l > eps) {
        double mid = (l + r) / 2;
        if (f(mid) > 0) {
            r = mid;
        } else {
            l = mid;
        }
    }
```

解方程求根：代码


```
19      return l;
20  }
```

目录

1 二分查找基础

2 二分查找变体

3 浮点数二分

4 二分答案

5 总结



什么是二分答案

二分答案：当直接求解答案十分困难，但**验证一个答案是否合法**很容易时，我们可以对“答案的值域”进行二分。

适用特征

- 答案具有单调性。例如：如果高度 H 满足条件，那么比 H 小的 H’ 也一定满足。
- 问题的解存在一个明确的上下界 `[l, r]`。
- 能够写一个 `check(x)` 函数，明确判定 `x` 是否满足条件。

解题步骤

1. 确定答案的单调性（求最大还是求最小）。
2. 设定二分的上下界 `l` 和 `r`。
3. 写出 `check(mid)` 函数，判断当前答案是否合法。
4. 根据 `check(mid)` 的结果，更新 `l` 或 `r`。

二分答案 = 把最优化问题，转化为判定问题

例题：砍树（整数二分答案）

题目：有 n 棵树，第 i 棵树的高度为 a_i。伐木工需要得到总量至少为 M 的木材。他设置一个伐木高度 H，所有高于 H 的树会被砍掉上面的部分，低于等于 H 的部分保留。求伐木工能达到的最大高度 H。

单调性提示：伐木高度 H 越低，砍下的木材总量越多；H 越高，木材总量越少。因此木材总量随 H 单调递减。

示例 1

- 输入：n $= 4$, M $= 7$, 树高分别为 20, 15, 10, 17
- 输出：15（当 H=15 时，砍下 $5 + 0 + 0 + 2 = 7$ 木材，刚好满足）

砍树：解法

四步法

- **状态（答案变量）**：二分的变量是伐木高度 `H`。
- **检查（check）**：当高度为 `h` 时，求所有树砍下的木材总长。如果 $\geq M$，返回 true。
- **边界**：下界 `l = 0`，上界 `r = max(a[i])`。
- **顺序**：求最大高度，所以如果 `check(mid)` 成立，尝试升高高度，即 `l = mid + 1`。

参考代码

砍树：解法


```cpp
bool check(int h) {
    long long s = 0;
    for (int i = 1; i <= n; i++) {
        s += max(0, a[i] - h);
    }
    return s >= M;
}

int l = 0, r = mx, best = -1;
while (l <= r) {
    int mid = (l + r) >> 1;
    if (check(mid)) {
        best = mid;
        l = mid + 1;
    } else {
        r = mid - 1;
    }
}
```

例题：切绳子（浮点二分答案）

题目：有 n 根绳子，第 i 根的长度为 a_i。现在要从中剪出 k 根长度相同的绳子，求这 k 根绳子的最大长度（保留 2 位小数）。

单调性提示：

- 目标长度 L 越长，每根绳子能切出的段数就越少。
- 这是一个典型的“随答案增加，条件变苛刻”的问题，满足单调递减性。

示例

- 输入：n = 3, k = 4, 绳子长度 3.0, 4.0, 5.0
- 输出：2.50（每根都切 2.5，3/2.5=1 段，4/2.5=1 段，5/2.5=2 段，总计 4 段）

切绳子：解法

核心思路：二分目标长度 L，检查所有的绳子是否能够切出至少 k 段。

关键步骤

- 二分的下界 `l = 0`，上界 `r =` 最长的绳子。
- `check(mid)` 函数：遍历所有绳子，累加 `(int)(a[i] / mid)`，如果 $\geq k$ 返回 true。
- 浮点循环条件：`while (r - l > 1e-4)`（保留 2 位小数精度通常取 1e-4）。
- 更新边界：如果 `check(mid)` 成立，说明还能切更长，更新 `l = mid`；否则 `r = mid`。

把切绳子的可行性判断转化为二分逻辑

切绳子：代码


```cpp
bool check(double len) {
    int cnt = 0;
    for (int i = 1; i <= n; i++) {
        cnt += (int)(a[i] / len);
    }
    return cnt >= k;
}

double l = 0, r = max_len;
while (r - l > 1e-4) {        // 此处直接用 1e-4 或全局 eps *
    double mid = (l + r) / 2;
    if (check(mid)) {
        l = mid;              // 还能切更长
    } else {
        r = mid;              // 切不出这么长
    }
}
printf("%.2f\n", l);
```

目录

1 二分查找基础

2 二分查找变体

3 浮点数二分

4 二分答案

5 总结



总结：二分算法的四要素

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

1. **单调性**：数据或答案必须存在单调关系，否则无法二分。
2. **上下界**：初始化 `l` 和 `r`，确保答案一定在这个区间内。
3. **mid 计算**：整数通常用 `(l + r) >> 1`，浮点数用 `(l + r) / 2`。
4. **条件判断**：依据 `check(mid)` 的结果，决定向哪半边收缩。

常见易错点

- 整数二分死循环：区间为 `[l, r]` 时，更新为 `l = mid + 1` 和 `r = mid - 1`。
- 浮点二分死循环：不写 `l <= r`，而是写 `r - l > eps`，或者固定跑 100 次。
- 浮点精度问题：判断相等用 `fabs(a - b) < eps`，不要直接用 `==`。
- 二分答案累加溢出：把 `s += a[i]` 中的 `s` 设为 `long long`。

整数盯边界，实数盯精度，答案找 check

附录：二分常见陷阱速查

错误	修复
整数死循环	检查 `l` 和 `r` 的更新是否包含 `mid` 本身
浮点死循环	检查循环条件是否写成了 `l <= r`，应改为 `r - l > eps`
浮点判断相等	严禁用 `==`，必须用 `fabs(a - b) < eps`
累加溢出	把 `s += a[i]` 中的 `s` 设为 `long long`

边界更新要彻底，check 条件要严密